1 Tests

Cette partie vérifie que `gaussp` donne les mêmes résultats que le programme séquentiel `gauss`, mesure ses performances sur une machine, puis étudie son comportement sur des entrées incorrectes. Toutes les valeurs ci-dessous ont été mesurées le 23 octobre 2026. Les commandes et sorties brutes sont conservées dans `tests/logs/`.

1.1 Conditions des essais

- Machine : `pc5201l` (20 cœurs logiques, 31 Go de mémoire), compte `safarl`, PVM 3.4.6 avec un seul hôte.
- `gauss` et `gaussp` sont compilés avec le même Makefile, sans option d'optimisation.
- Le chronométrage sépare le chargement et le calcul ; la sauvegarde est exclue des deux.
- Dans la version de départ, l'appel `gauss(tab, N, P, me, tids)` était commenté dans `dowork()`. Il a été réactivé avant les tests : sans lui, aucun calcul n'est effectué.
- La machine est partagée : un démon PVM d'un autre utilisateur y tournait pendant les essais (charge moyenne proche de 1 au départ).

Matrices : `m4`, `m8`, `m16`, `m500` et `m1000` sont les fichiers personnels (identiques dans `matrices-test/` et `bin/LINUX64/`, mêmes sommes MD5). `m2000` a été générée par `tests/genmat.py` (diagonale ≈ 40 000, autres coefficients entre −10 et 10). `m5000` est le fichier déjà présent dans le dossier. Ces fichiers sont identiques à ceux du dépôt du projet (`matrices-test/`). Les matrices d'erreur proviennent de `matrices-test/erreurs/` dans le dépôt, sauf `m4-non-numerique` (copie de `m4` où un coefficient est remplacé par `abc`).

1.2 Validation du calcul

Pour chaque cas, `gauss` est lancé, son résultat est copié dans `<matrice>.seq.result`, puis `gaussp` est lancé depuis la console PVM :

```
$ ./gauss 1000 m1000
$ cp m1000.result m1000.seq.result
pvm> spawn -4 -> gaussp 1000 /user/safarl/pvm3/bin/LINUX64/m1000
$ tests/compare.py m1000.seq.result m1000.result 1000
```

Le script `compare.py` vérifie le nombre de valeurs (N × N), l'absence de `nan` ou de valeur infinie, compte les écarts supérieurs à 0,011 et relève le maximum absolu sous la diagonale.

Matrice	P testés	Valeurs	Écarts > 0,011	Max. sous la diagonale
`m4`	1, 2, 4	16	0	0,00
`m8`	2, 4	64	0	0,00
`m16`	1, 2, 4, 8	256	0	0,00
`m500`	4	250 000	0	0,00
`m1000`	4	1 000 000	0	0,00
`m2000`	4	4 000 000	0	0,00

Tableau 1. – Comparaison entre gauss et gaussp : résultats identiques dans les 12 configurations.

Dans tous les cas, l'écart maximal est de 0,00 : les fichiers sont identiques au centième près. La matrice obtenue est bien triangulaire. Cette vérification porte sur des valeurs arrondies à deux décimales.

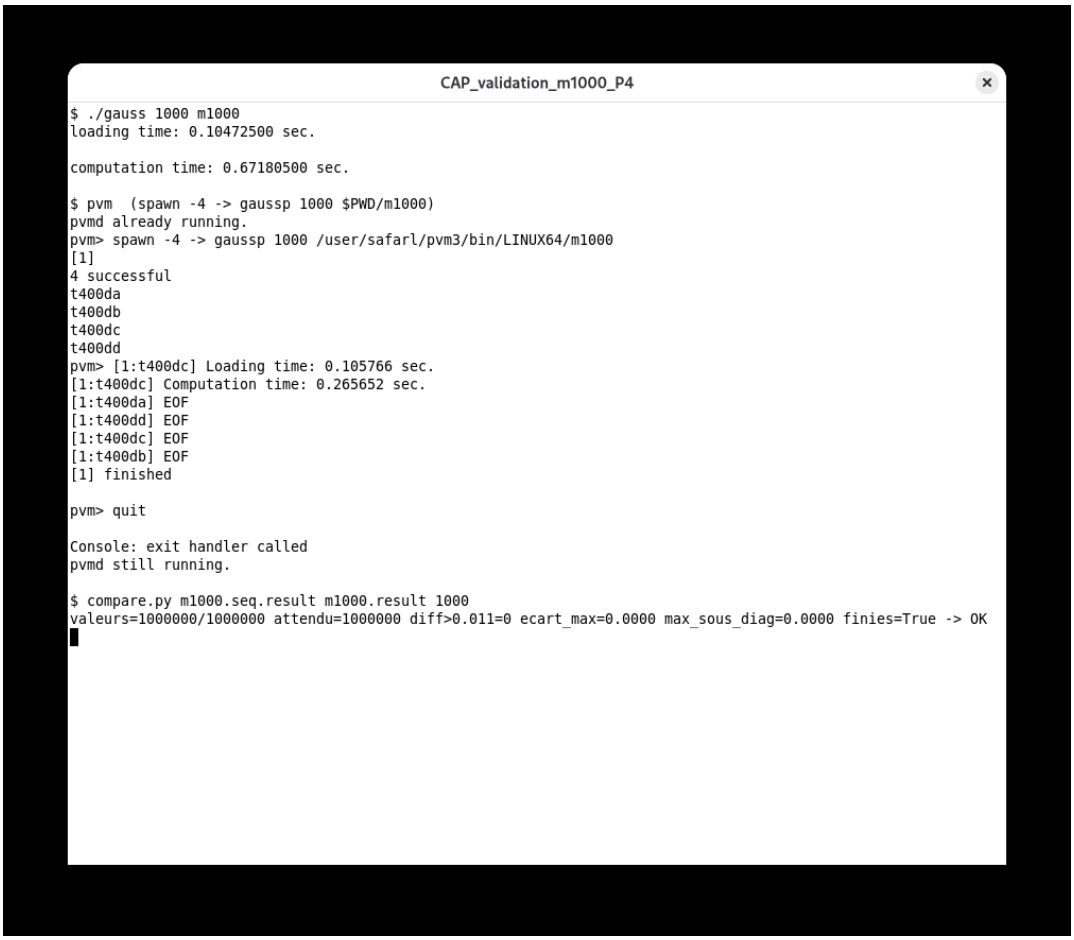


Fig. 1. – Exécution séquentielle, parallèle (P = 4) et comparaison sur `m1000`.

1.3 Mesures de performance

1.3.1 Protocole

Cinq exécutions par configuration pour N = 500 et N = 1 000, trois pour N = 2 000, une seule pour N = 5 000. Les essais sont lancés l'un après l'autre par `tests/perf.sh`. On retient la médiane du temps de calcul, qui inclut les échanges des lignes pivots et la barrière finale.

$$S(P) = \frac{T_{\text{seq}}}{T_{\text{par}}}(P) \qquad E(P) = \frac{S(P)}{P}$$

Une configuration n'est testée que si P divise N (P = 8 est donc exclu pour N = 500).

1.3.2 Résultats

Version	N = 500	N = 1 000	N = 2 000	N = 5 000[1]
Séquentielle	0,0836	0,677	5,26	80,7
P = 1	0,0745	0,596	4,66	—
P = 2	0,0491	0,376	2,56	—
P = 4	0,0326	0,214	1,43	19,6
P = 8	—	0,188	1,54	—
P = 10	0,0332	0,186	1,15	14,7
P = 20	0,0501	0,229	1,10	14,2

Tableau 2. – Temps de calcul médians en secondes. [1] Un seul essai.

P	N = 500	N = 1 000	N = 2 000	N = 5 000
1	1,12 (112 %)	1,14 (114 %)	1,13 (113 %)	—
2	1,70 (85 %)	1,80 (90 %)	2,06 (103 %)	—
4	**2,56** (64 %)	3,17 (79 %)	3,67 (92 %)	4,12 (103 %)
8	—	3,59 (45 %)	3,42 (43 %)	—
10	2,52 (25 %)	**3,64** (36 %)	4,56 (46 %)	5,51 (55 %)
20	1,67 (8 %)	2,95 (15 %)	**4,80** (24 %)	**5,68** (28 %)

Tableau 3. – Accélération S(P) et efficacité E(P) par rapport au séquentiel.

Les mesures brutes (toutes les répétitions, minimum et maximum) se trouvent dans `tests/logs/mesures_brutes.csv` et `tests/logs/synthese.csv`.

1.3.3 Analyse

Le temps diminue avec P, jusqu'à un certain point. Pour N = 500, le gain plafonne dès P = 4 (S = 2,56) puis se dégrade : avec P = 20, le calcul est plus lent qu'avec P = 4. Pour N = 1 000, le gain plafonne entre P = 8 et P = 10 (S ≈ 3,6) et redescend à P = 20.

Les grandes matrices profitent davantage du parallélisme. L'accélération maximale passe de 2,56 (N = 500) à 3,64 (N = 1 000), 4,80 (N = 2 000) et 5,68 (N = 5 000). Quand N augmente, chaque tâche fait plus de calcul entre deux réceptions de pivot.

P = 1 est plus rapide que le séquentiel d'environ 12 %, de façon régulière sur toutes les tailles. Le gain ne vient donc pas du parallélisme : les deux programmes n'écrivent pas les boucles et les calculs d'indices de la même façon. Ce point explique aussi les efficacités supérieures à 100 % (N = 2 000, P = 2). Par rapport à P = 1, l'accélération de N = 2 000 avec P = 2 est de 1,82.

Anomalie à P = 8 pour N = 2 000 : 1,54 s, plus lent que P = 4 (1,43 s). Sur seulement trois essais, cela peut venir de la charge de la machine ou du placement des processus ; il faudrait refaire la mesure pour conclure.

Variabilité : le séquentiel est très stable (moins de 1 % d'écart). Les versions parallèles ont quelques essais isolés nettement plus lents : 0,0680 s au lieu de 0,0491 s (N = 500, P = 2), 0,261 s au lieu de 0,214 s (N = 1 000, P = 4). La médiane écarte ces valeurs.

Le chargement n'est pas accéléré. P0 lit tout le fichier puis envoie les lignes. Pour N = 5 000, le chargement prend 3,1 à 3,6 s en parallèle contre 2,4 s en séquentiel. Sur le temps total (chargement + calcul), on passe tout de même de 83 s à environ 18 s avec P = 20.

Ces explications (coût des communications, attentes, nombre de cœurs) sont des hypothèses : les temps seuls ne permettent pas de les démontrer.

1.4 Variante : n'envoyer que la partie utile du pivot

À l'étape k, les colonnes 0 à k–1 de la ligne pivot sont déjà nulles et inutiles aux autres tâches. La variante `gaussp_utile.c` n'envoie que les colonnes k à N–1 (3 lignes modifiées).

N	P	Ligne complète (s)	Partie utile (s)	Gain
500	4	0,0340	0,0270	21 %
1 000	4	0,224	0,201	11 %
1 000	8	0,188	0,172	8 %
2 000	4	1,449	1,380	5 %

Tableau 4. – Médiane de 3 essais. Résultats identiques entre les deux versions.

Le gain est réel mais modeste, et il diminue quand N augmente : le volume de messages est divisé par deux environ, mais le calcul reste prépondérant sur les grandes matrices.

1.5 Observation avec XPVM

`gaussp` a été lancé depuis XPVM avec traçage (N = 16, P = 4). La trace complète est sauvegardée dans `tests/logs/xpvm_m16_P4.trace`.

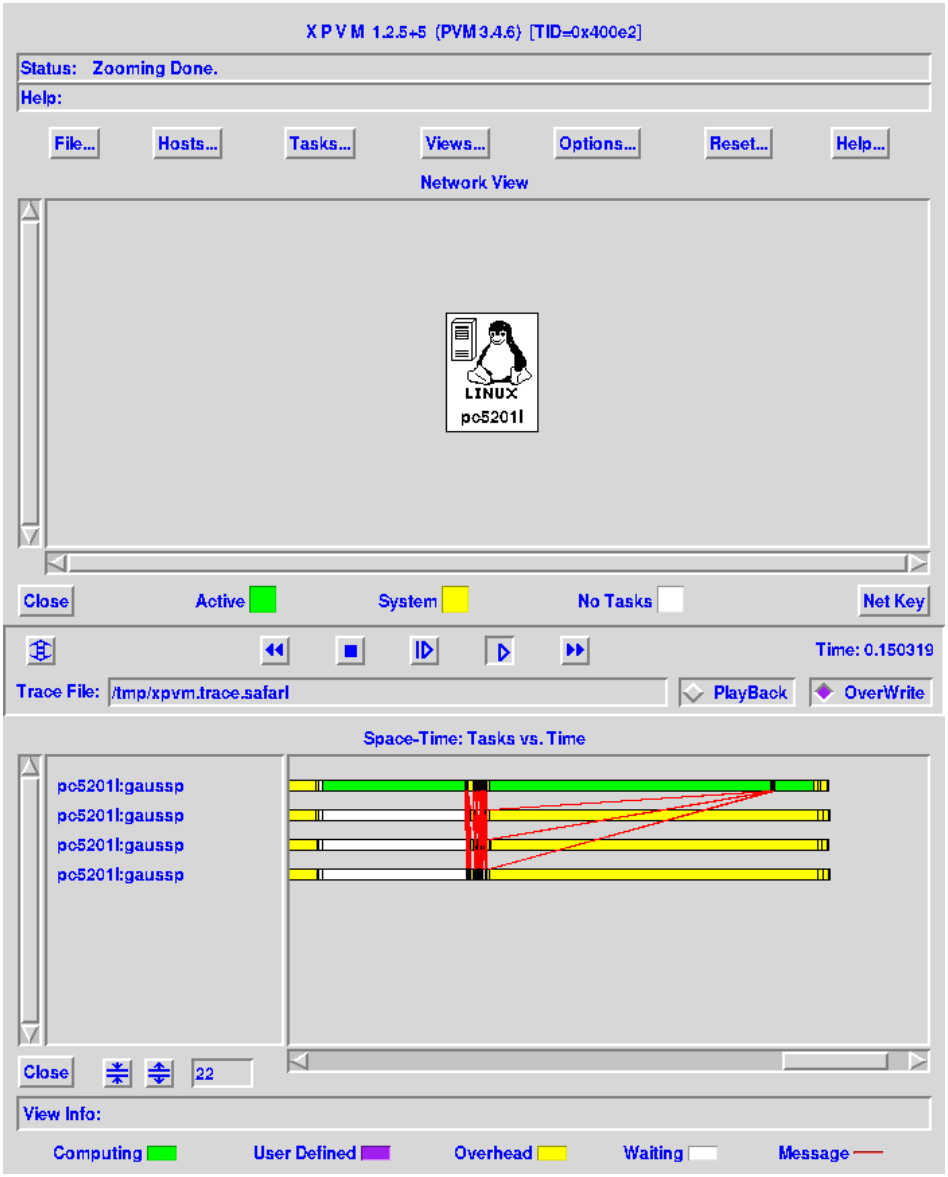


Fig. 2. – Vue Space-Time de XPVM : N = 16, P = 4 (zoom sur les échanges).

On distingue sur la trace :

- au début, P0 (première ligne) calcule seul (vert) pendant que les trois autres tâches attendent (blanc) : c'est la lecture du fichier et la distribution des lignes ;
- un bloc dense de messages (rouge) : la distribution puis les 15 diffusions de lignes pivots, très rapides pour N = 16 ;
- à la fin, des messages qui convergent vers P0 : le rassemblement des lignes pour la sauvegarde ;
- de longues zones jaunes (surcoût PVM) liées aux barrières et à la sortie du groupe.

Sur une matrice aussi petite, le calcul est négligeable devant les communications et la synchronisation.

1.6 Tests des cas d'erreur

Chaque cas est lancé par `tests/erreurs.sh`. Après 15 s, la commande `ps -a` de la console indique les tâches restantes, qui sont ensuite supprimées avec `reset`. L'ancien fichier `.result` est effacé avant chaque essai.

Cas (P)	Message observé	Tâches bloquées	Fichier .result
Fichier inexistant (4)	`matrix_load : fopen: No such file or directory`	3	Non
Fichier incomplet, 14/16 valeurs (2)	Aucun ; dernier coefficient faux (−0,57)	0	Oui, faux
Coefficient non numérique (2)	`pivot 2 presque nul` : lecture arrêtée sur `abc`	1	Non
Pivot nul au départ (2)	`ATTENTION: pivot 0 presque nul: 0`	1	Non
Pivot nul pendant le calcul (2)	`ATTENTION: pivot 1 presque nul: 0`	1	Non
N = 6 non divisible par P = 4	Aucun	4	Vide
N = 0 (2)	Aucun	0	Vide
N = −4 (2)	Aucun	0	Vide
Arguments manquants (2)	Aucun, les tâches s'arrêtent	0	Non
Sortie non inscriptible (2)	`matrix_save : fopen: Permission denied`	0	Non

Tableau 5. – Comportement observé de la version testée.

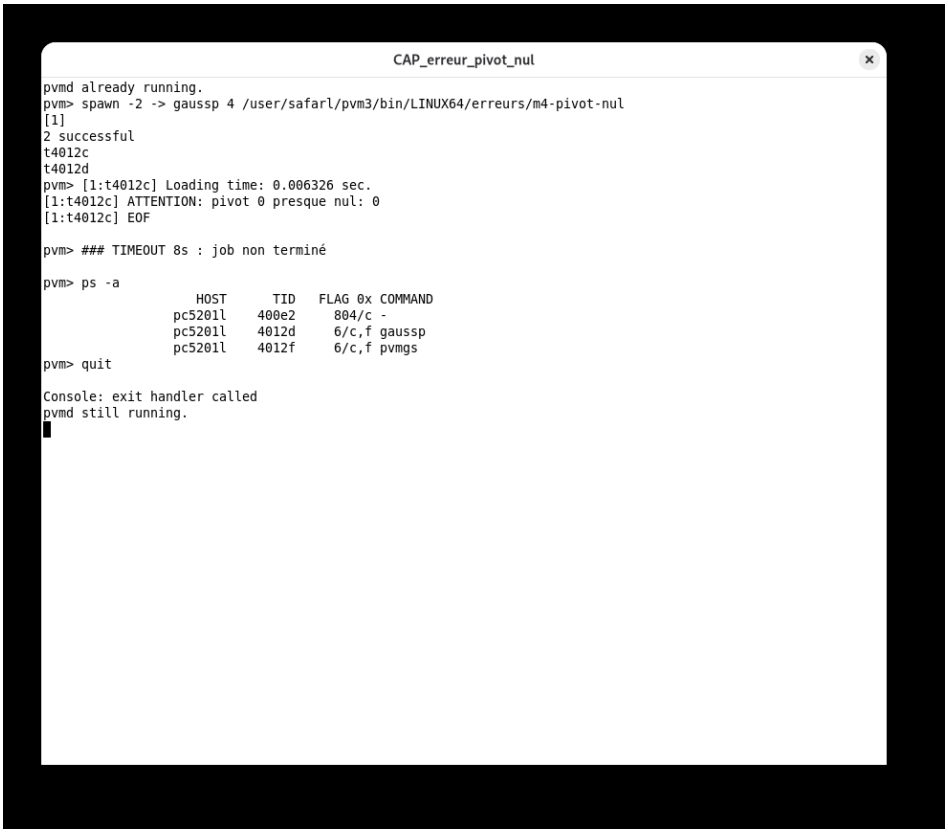


Fig. 3. – Pivot nul : P0 affiche le message et s'arrête, l'autre tâche `gaussp` reste bloquée.

Constats :

- quand une tâche s'arrête avec `exit()` (pivot nul, fichier absent), les autres restent bloquées dans `pvm_recv` ou `pvm_barrier` et doivent être tuées à la main ;
- un fichier incomplet ou un coefficient non numérique n'est pas détecté, car la valeur de retour de `fscanf` n'est pas vérifiée. Dans le cas `abc`, l'erreur n'est signalée que par chance, via un pivot nul ;
- N n'est pas contrôlé : non divisible par P, le programme se bloque ; nul ou négatif, il se termine sans message avec un fichier vide.

Le séquentiel `gauss` a les mêmes faiblesses de lecture ; sur un fichier inexistant, il plante même avec une erreur de segmentation (code retour 139), car `fclose` est appelé sur un pointeur nul.

Corrections possibles : vérifier `argc`, `N > 0` et `N % P == 0` avant tout calcul ; tester le retour de `fscanf` ; en cas d'erreur, prévenir les autres tâches (message dédié ou `pvm_kill`) au lieu d'un simple `exit()`.